M2103 PROGRAMMATION PAR OBJETS

TD 1. OBJETS

OBJETS ET CLASSES

- Objet : entité qui rend un service
  - État interne : des attributs
  - Utilisation : son API (méthodes appelables)
- Classe : décrit des objets (instances)

EXEMPLE DE CLASSE

- (Noter : *var*)
- **Question** : quel est le service rendu par les objets de cette classe ?

```
class Powers {
   double next;
   double factor;

   Powers(double initial, double factor) {
       this.factor = factor;   // this = cette instance
       next = initial;
   }

   double getNext() {
       var result = next;
       next *= factor;
       return result;
   }
}
```

EXEMPLE DE CLASSE

- Créer des instances :

```
var covid = new Powers(1.0, 1.04);
```

- Appeler des méthodes :

```
for (int i = 1; i <= 5; i++) {
    System.out.println(covid.next())
}
```

```
class Powers {
    double next;
    double factor;

    Powers(double initial, double factor) {
        this.factor = factor;    // this = cette instance
        next = initial;
    }

    double getNext() {
        var result = next;
        next *= factor;
        return result;
    }
}
```

CLASSE DATE : BESOINS

- On saute les difficultés (heure d'été, années bissextiles, etc.)
- **Qu'est-ce qu'on pourrait faire avec cette classe ?** (à droite).

```
var bastille = new Date(14, 7, 1789);

System.out.println("Prise Bastille le "
    + bastille.getDay()
    + "/" + bastille.getMonth());

Date[] dates = {
    bastille,
    new Date(2, 12, 1805),
    new Date(13, 7, 1515)
};

Arrays.sort(dates);

for (var date : dates) {
    System.out.println("-> " + date);
}
```

CLASSE DATE : L'API

- Donc il nous faut :
- De quoi construire une nouvelle date : **constructeur** Date(jour, mois, année)
- De quoi récupérer le jour, le mois, l'année : **accesseurs** getDay(), getMonth(), getYear()
- De quoi trier des dates : **comparateur** compareTo(date)
- De quoi convertir les dates en texte pour l'affichage : toString()

```
var bastille = new Date(14, 7, 1789);

System.out.println("Prise Bastille le "
    + bastille.getDay()
    + "/" + bastille.getMonth());

Date[] dates = {
    bastille,
    new Date(2, 12, 1805),
    new Date(13, 7, 1515)
};

Arrays.sort(dates);

for (var date : dates) {
    System.out.println("-> " + date);
}
```

CLASSE DATE : IMMUABILITÉ

- Sémantique de valeurs : une date n'est jamais modifiée.

```
var date = new Date(15, 1, 2021);
var rdvDentiste = date;
var rdvPlombier = date;
```

- Utile pour ne pas modifier un objet qui a plusieurs références…

```
var rdvDentiste = new Date(23, 1, 2021);
```

- Rappel : String (et Integer…) donne des objets immuables.

CLASSE DATE : TESTS

- **Exercice** : poser quelques tests unitaires sur les méthodes de l'API.

CLASSE DATE : BASE DE LA DÉFINITION

- Sans se poser de questions (l'immuabilité est assurée par *final*) :

```
class Date {
   final int day, month, year;

   Date(int day,int month, int year) {
      this.day   = day;
      this.month = month;
      this.year  = year;
   }
   ...
}
```

- L'IDE (Netbeans) peut engendrer les méthodes *get* tout seul (Ctrl-Insert).

CLASSE DATE : COMPARAISON DES DATES

- En Java, les objets comparables doivent **respecter** Comparable :

```
class Date implements Comparable<Date> {
    ...
}
```

- Pour cela, il faut ajouter une méthode compareTo(other) qui renvoie trois valeurs possibles (entiers) :
  - négatif si l'objet courant (this) vient avant l'autre (en paramètre),
  - positif s'il vient après,
  - nul (0) en cas d'égalité.

CLASSE DATE : COMPARAISON DES DATES

- Pour les dates, une représentation comme 17890714 fonctionne (AAAAMMJJ) avec la soustraction :

```
int asNumber() {
    return day + 100*month + 10_000*year;
}
```

- Ensuite :

```
@Override
public int compareTo(Date other) {
    return this.asNumber() - other.asNumber();
}
```

NOTATIONS : PLUS TARD…

- Visibilité (public, private, etc.) : par la suite (et très vite en conception)
  - Ici : on laisse tout en *package private* (par défaut), dont asNumber() qui ne fait pas partie de l'API (usage interne)
- Respecter un contrat (comparable) en Java : implements, override
  - Permet d'appeler des méthodes classiques dont sort pour trier
  - Vos propres contrats (interfaces) : par la suite

EXEMPLE : PLANNING

- On souhaite stocker plusieurs *tâches*, qui sont à faire entre deux dates, par exemple :

```
class Task {

    final String description;
    final Date startDate, endDate;

    Task(String description, Date startDate, Date endDate)  {
        ...
    }
    ...
}
```

PLANNING : API

- On voudrait faire ceci :

```
Planning p = new Planning();

p.addTask("Préparer les cours",
          new Date(20,12,2020),
          new Date(3,2, 2021));
//...

var todo =  p.getTasks(new Date(25, 1, 2021))) {

for (var task : todo) {
    System.out.println(task);
}
```

- Ce qui implique de stocker plusieurs tâches dans le planning.

PLANNING : HASHSET

- HashSet permet de représenter un ensemble d'objets, ici un ensemble de tâches.

```
class Planning {
    HashSet<Task> tasks;

    Planning() {
       tasks = new HashSet<>();
    }

    void addTask(String description, Date startDate, Date endDate) {
       Task task = new Task(description, startDate, endDate);
       tasks.add(task);
    }

//...
```

- Ici, le planning crée chaque tâche (construit, alloue) à partir de données.

PLANNING : HASHSET

- Méthodes disponibles sur HashSet :
  - add (ajouter si non présent)
  - remove (retirer si présent)
  - contains (vérifie si présent)
  - size (combien d'éléments)
- Plus boucle for sur tous les éléments (ordre non garanti ici)

```
class Planning {
    HashSet<Task> tasks;

    Planning() {
       tasks = new HashSet<>();
    }

    void addTask(String description, Date startDate, Date endDate) {
       Task task = new Task(description, startDate, endDate);
       tasks.add(task);
    }

//...
```

PLANNING : ARRAYLIST

- Un ensemble n'a pas d'ordre, getTasks devrait ordonner les tâches pour être utile (**comment ?**)
- Séquence avec ordre : tableau ou liste, ici *ArrayList* (qui est les deux) avec tri au choix du client :

```
var todo =  p.getTasks(new Date(25, 1, 2021))) {

// tri par proximité de la fin
todo.sort(Comparator.comparing(Task::getEnd));

for (var task : todo) {
    System.out.println(task);
}
```

PLANNING : ARRAYLIST

- Code de la méthode getTask :

```
ArrayList<Task> getTasks(Date date) {
    var list = new ArrayList();
    for (var task : tasks) {
        if (task.compareTo(date) == 0) {
            list.add(task);
        }
    }
    return list;
}
```

PLANNING : ARRAYLIST

- ArrayList dispose de méthodes : add, contains, remove, size…
- Mais c’est un tableau (avec indices de 0 à taille-1), accès avec get(i)
- Donc ajouter ou supprimer au milieu implique un décalage, la recherche un parcours… (et c’est plus lent).

NOTIONS OBJET

- Un objet est une entité qui rend des services
- Individu (entité) : attributs modifiables (l'objet évolue), valeur (immuable) : attributs définitifs (l'objet représente une valeur unique)
- Les objets sont définis par des classes
- Créer une classe :
  - Comment utiliser les objets de cette classe ? Exemples d'exécution, tests
  - Déduire l'API (services fournis = méthodes appelables), coder la représentation des données nécessaires (uniquement) puis des méthodes (avec méthodes auxiliaires si nécessaire)

AUTRES ÉLÉMENTS

- *var* : type automatique Java
- Comparable : permet de faire des comparaisons entre objets, Comparator contient des comparaisons prédéfinies
- Collections : ensembles, listes… plutôt que les tableaux tab[]
- Types génériques : paramétrés par une classe A<T>
- Définir une opération imposée : Override (equals, toString, compareTo)
- Tous les détails plus tard dans ce module, en conception, et vite en TP.